

AI Project 1 and 2

Word2Vec

Yue Lin - 2025.4.3

Overview

- There will be 4 projects in total:
 - Project 1 and 2: Word2Vec (TA: Yue Lin)
 - Project 1: CBOW; Project 2: Skip-Gram
 - (We will continue to use materials from previous years, but GLoVe will not be included.)
 - Project 3: Transformer (TA: Haoran Li)
 - Project 4: BERT (TA: Haoran Li)

Word2Vec

- We want to learn a mapping from each word to a vector.
 - The vector should capture certain semantic properties of the word.
 - Words that are semantically similar should be mapped to vectors that are close to each other, under some distance metric.
 - E.g. good v.s. excellent
 - It should support analogy reasoning to some extent: king - man + woman $\approx$ queen

Word2Vec

- We learn word semantics from context.
 - We feed the model with a large number of sentences.
 - The sentence “I want to drink tea” appears xxx times.
 - The sentence “I want to drink coffee” appears xxx times.
 - From this, the model learns that “tea” and “coffee” often occur in similar contexts, and thus are semantically similar to some extent.

Word2Vec

Self-Supervised Learning

- We have tons of sentences. “The dog barks loudly at night.”
 - We “mask” some words and let our model to guess it.
 - model_input: “The __ barks loudly at night.”
 - label: “dog”
 - loss: loss_function(“dog”, model_output)
- The more frequently it appears, the more often the updates will move in that direction.

Word2Vec

CBOW and Skip-Gram

- “The dog barks loudly at night.”
- CBOW: context -> word
 - model_input: ["The", "barks", "loudly", "at"]
 - label: “dog”
- Skip-Gram: word -> context
 - model_input: “dog”
 - label: ["The", "barks", "loudly", "at"]

Purposes

- Help you gain hands-on experience with a simple but effective neural model for learning word embeddings
- Strengthen your understanding of basic NLP preprocessing using tools like NLTK
- Prepare you for upcoming projects that build upon these word embedding foundations

Content

- CBOW.pdf, Skip-gram-GloVe.pdf
- Prerequisite:
 - Install Anaconda (Optional)
 - Install PyTorch
 - Install NLTK (Natural Language Toolkit)
- Clone and run the CBOW code from the following repository:
 - <https://github.com/HIT-SCIR/plm-nlp-code>
- Evaluate the performance of the model.

Training CBOW

- On an Apple M4 Pro chip, training for 10 epochs takes about 15 min.

```
• (basic) (base) yue@Mac project1and2_yue % /opt/anaconda3/envs/basic/bin/python /Users/yue/Documents/P
hD/TA/CSC6021_AI/project/project1and2_yue/plm-nlp-code-main/chp5/cbow.py
Dataset Construction: 100%|████████████████████| 54716/54716 [00:01<00:00, 46275.42it/s]
Training Epoch 0: 100%|████████████████████| 1574/1574 [01:03<00:00, 24.76it/s]
Loss: 10209.36
Training Epoch 1: 100%|████████████████████| 1574/1574 [01:04<00:00, 24.42it/s]
Loss: 8268.40
Training Epoch 2: 100%|████████████████████| 1574/1574 [01:03<00:00, 24.65it/s]
Loss: 7713.15
Training Epoch 3: 100%|████████████████████| 1574/1574 [01:03<00:00, 24.70it/s]
Loss: 7377.67
Training Epoch 4: 100%|████████████████████| 1574/1574 [01:02<00:00, 25.05it/s]
Loss: 7137.86
Training Epoch 5: 100%|████████████████████| 1574/1574 [01:03<00:00, 24.87it/s]
Loss: 6951.37
Training Epoch 6: 100%|████████████████████| 1574/1574 [01:02<00:00, 25.24it/s]
Loss: 6798.56
Training Epoch 7: 100%|████████████████████| 1574/1574 [01:03<00:00, 24.72it/s]
Loss: 6669.69
Training Epoch 8: 100%|████████████████████| 1574/1574 [01:03<00:00, 24.97it/s]
Loss: 6558.23
Training Epoch 9: 100%|████████████████████| 1574/1574 [01:03<00:00, 24.96it/s]
Loss: 6459.91
Pretrained embeddings saved to: cbow.vec
```


Evaluation

- Project 1, 2 Evaluations.pdf
- We need to evaluate the performance of the model.
- Evaluation:
 - K-Nearest Neighbor
 - Word similarity golden standard evaluation
 - Analogy reasoning evaluation

Evaluation: KNN

- You are provided with a set of 50 words.
- Compute the embeddings of all words using the trained model.
- For each word:
 - Identify the top-k most similar words based on cosine similarity.
 - $k=10$
- Evaluation: Perform a subjective assessment to determine whether the retrieved nearest neighbors are truly semantically similar to the original word.

Evaluation: KNN

```
# query word list
word_list = ["july", "reliable", "play", "willing", "good", "very", "patient", "co
"generally", "gradually", "happy", "able", "close", "near",
              "saturday", "friend", "company", "road", "plane", "war", "politics",
              "china", "experience", "police",
              "give", "create", "tell", "become", "lack", "win", "help", "gain", "g
              "difficult", "go", "man", "ten", "year"]

# calculate cosine similarity
def cos():
    # TODO
    pass

# find k-nearest neighbors
def knn():
    # TODO
    pass

# evaluation process for each word
def evaluate(word):
    knn_words = []
    knn_sims = []
    # TODO
    print("\nQuery:", word)
    for i in range(len(knn_words)):
        print("The similarity of {} is {}".format(knn_words[i], knn_sims[i]))

# TODO
for word in word_list:
    evaluate(word)
```

- K-Nearest Neighbour template.py
- Complete the evaluation code.

Evaluation: KNN

My Result: CBOW

- Query: good
- significant 0.569.
- promising 0.526.
- relevance 0.504.
- preparations 0.500. Why this word "preparation" is close to "good" in semantics?
- satisfactory 0.499.
- Average similarity for good is 0.519.

Write a paragraph discuss the results.

Evaluation: KNN

My Result: Skip-Gram

- Query: good
- underestimated 0.559 **Why "underestimated" is close to "good"?**
- sustainable 0.551. **Why?**
- satisfactory 0.530.
- unsatisfactory 0.525. (Why?)
- favors 0.521. **Why?**
- Average similarity for good is 0.537.
- (Please explain the results.) **Although "good" and its KNN's avg similarity is 0.537 for Skip-gram model, several KNN are questionable.**

- The requirement:
- Prof. Ding: KNN evaluation task 1
 - You are given a list of 50 words. For each word, find the top 10 KNN words. For each word, compute average similarity (word, top KNN). Then average similarity (word, top KNN) for all 50 words. [for CBOW or Skip-Gram model.](#)
 - Pick 4 words from above 50 words. Show the details in 4 figures. And give a discussion of the result. As the example in the next page.

Evaluation: KNN

One student Project 1 Report

```
2 Query: july
3 The similarity of april is 0.7744300961494446
4 The similarity of june is 0.7628009915351868
5 The similarity of october is 0.7144432663917542
6 The similarity of march is 0.6481677293777466
7 The similarity of august is 0.6417593359947205
8 The similarity of tomorrow is 0.5825945734977722
9 The similarity of 1984 is 0.5697567462921143
10 The similarity of november is 0.566382884979248
11 The similarity of january is 0.5634446144104004
12 The similarity of 1989 is 0.544720470905304
13 average similarity: 0.6368500709533691
```

Figure 1

```
67 Query: very
68 The similarity of extremely is 0.6010755300521851
69 The similarity of reasonably is 0.5556564331054688
70 The similarity of too is 0.5548741817474365
71 The similarity of larger is 0.5358619093894958
72 The similarity of quite is 0.5323862433433533
73 The similarity of relatively is 0.524463415145874
74 The similarity of severe is 0.49323877692222595
75 The similarity of fairly is 0.48921892046928406
76 The similarity of precisely is 0.4856249988079071
77 The similarity of less is 0.4846831262111664
78 average similarity: 0.5257083535194397
```

Figure 2

More examples demonstrate a similar performance result. Figure 2 shows the KNN for “very”. The words “extremely”, “reasonably” etc. are adverbs that describe the extent of an adjective, which are similar to “very”. Other words, however, are not very semantically similar to “very”, such as “larger”, “severe”, and “less”. Besides, we expect “quite” to be more similar than “larger”, and “fairly” to be more similar than “severe”. The model fails to demonstrate these semantics similarity relations.

Evaluation: KNN

Comment

- An example word:
 - CBOW results.
 - Skip-Gram results.
- Another example word:
 - CBOW results.
 - Skip-Gram results.
- ...

Evaluation: Word similarity golden standard evaluation

- A manually labeled word similarity dataset, SimLex-999, is provided.
 - Each entry: word 1, word 2, and similarity score assigned by experts.
 - This score can be regarded as the ground truth.
- Use the trained model to compute the cosine similarity for each word pair and compare it with the ground truth.
 - Then compute a Spearman correlation coefficient to measure how well the model's similarities align with the reference.
 - Randomly pick 20 word pairs to show the results.

Evaluation: Word similarity golden standard evaluation

My Result: Skip-Gram

- word1, word2, standard_similarity, cosine_similarity, scaled_similarity
- old, new, 1.58, 0.23, 6.13 (not as expected)
- smart, intelligent, 9.20, 0.11, 5.52
- hard, difficult, 8.77, 0.32, 6.59 (as expected)
- hard, easy, 0.95, 0.25, 6.24
- fast, rapid, 8.75, 0.25, 6.23 (as expected)
- **(Please write one paragraph to comment on the results of the 10 pairs.)**

How to normalize a cosine similarity to the range [0, 10]

- Given a word cosine similarity, we want to scale it to the range [0, 10]
- original range: $[-1, 1] \rightarrow [0, 10]$
- for x in original range, we compute $(x+1)*5$

Evaluation: Word similarity golden standard evaluation

My Result: CBOW

- word1, word2, standard_similarity, cosine_similarity, scaled_similarity
- old, new, 1.58, 0.11, 5.57
- smart, intelligent, 9.20, -0.04, 4.83
- hard, difficult, 8.77, 0.33, 6.67
- hard, easy, 0.95, 0.09, 5.46
- fast, rapid, 8.75, 0.25, 6.27
- **(Please list CBOW results first and skip-gram second. Make a comment on which is better. Put it together.)**

Evaluation: Word similarity golden standard evaluation

Comment

- Prof. Ding: Your comment should follow this loop
- do loop, for each word pair in the 20 word pairs:
 - Show CBOW results.
 - Write one paragraph to comment on the CBOW results.
 - Show Skip-Gram results.
 - Write one paragraph to comment on the Skip-Gram results.
 - Write one paragraph to comment on the performance comparison between the two models.
- end do

Evaluation Task 3: Analogy reasoning evaluation

- $\text{vector}(\text{queen}) \approx \text{vector}(\text{king}) - \text{vector}(\text{man}) + \text{vector}(\text{woman})$
- Use Mikolov's analogy dataset, where each line typically contains a word analogy in the format A B C D, representing the relationship A:B :: C:D.
 - Compute the predicted vector
$$D_{\text{pred}} = \text{embedding}(B) - \text{embedding}(A) + \text{embedding}(C)$$
This is a 64-dim vector
 - Given the embedding D_{pred} , find the predicted word.
search the closest (cosine) word in the list
 - See if the predicted word is exactly D.
- Calculate the accuracy.
You need to show (plot) D_{pred} and predicted word (search result), both are 64-dim vectors
- Randomly pick 10 examples and show the results.

Evaluation: Analogy reasoning evaluation

My Result: CBOW

- For 8591 analogy questions with all words presented in the vocabulary, 37 of them are correctly answered.
- Sample 5 analogy questions results:
- A:B :: C:D: athens : greece :: bangkok : thailand | Predicted: peking | Correct: False
- A:B :: C:D: athens : greece :: beijing : china | Predicted: elmd | Correct: False
- A:B :: C:D: athens : greece :: berlin : germany | Predicted: mcs | Correct: False
- A:B :: C:D: athens : greece :: cairo : egypt | Predicted: zimmer | Correct: False
- A:B :: C:D: athens : greece :: canberra : australia | Predicted: telex | Correct: False

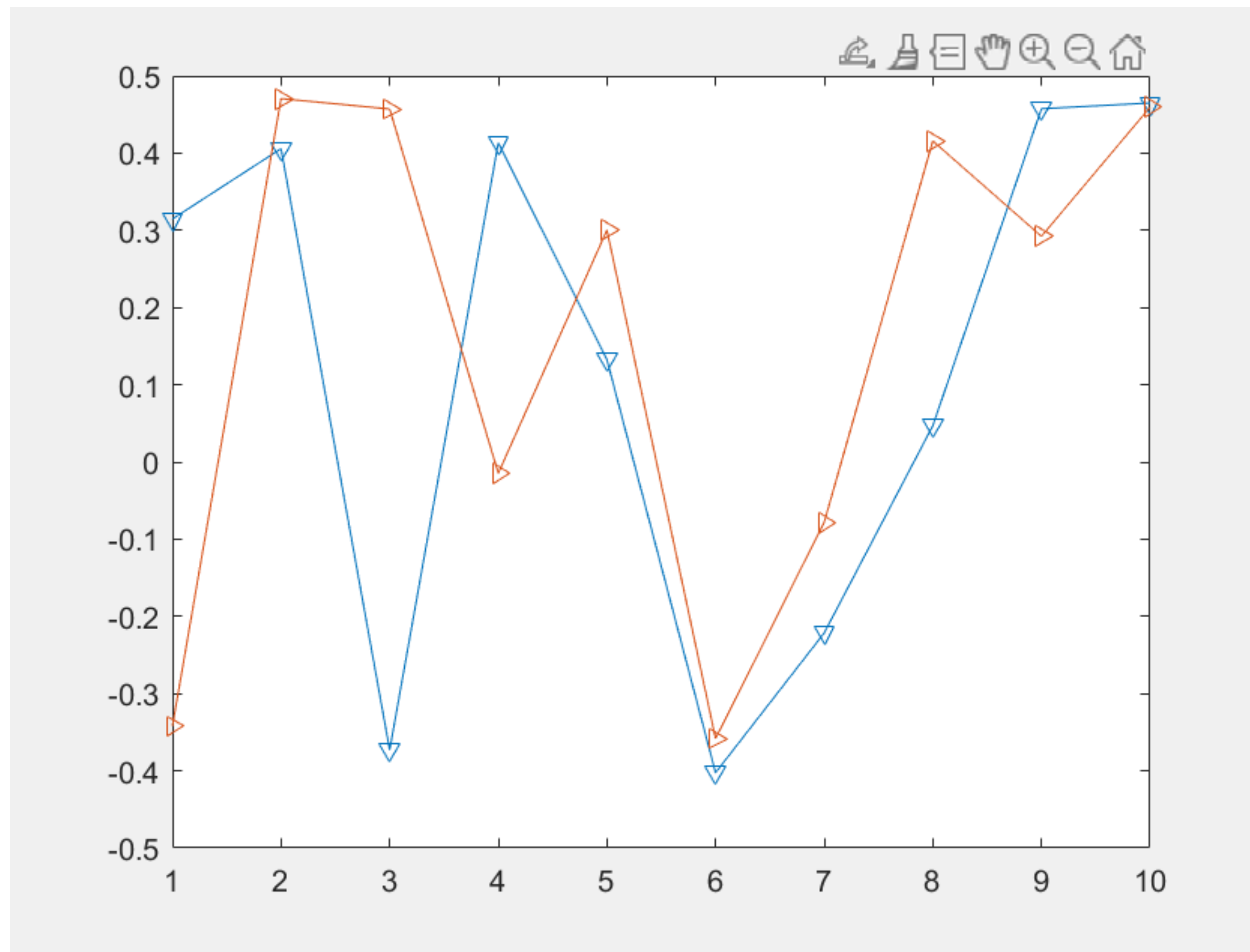
Evaluation: Analogy reasoning evaluation

My Result: Skip-Gram

- For 8591 analogy questions with all words presented in the vocabulary, 42 of them are correctly answered.
- Sample 5 analogy questions results:
- A:B :: C:D: enhance : enhances :: see : sees | Predicted: lint | Correct: False
- A:B :: C:D: go : goes :: provide : provides | Predicted: soyproducts | Correct: False
- A:B :: C:D: seattle : washington :: toledo : ohio | Predicted: obstructing | Correct: False
- A:B :: C:D: long : longest :: slow : slowest | Predicted: mainly | Correct: False
- **A:B :: C:D: Moscow : Russia :: Beijing : China | Predicted: China | Correct: True**
- (For each category, pick one pair and find the predicted vector D_pred.)

Evaluation: Analogy reasoning evaluation

Plot



- Plot the two embedding vectors
 - The predicted vector D_{pred}
 - The search result (The most similar one)
- X-axis is the 64 dimensions of the embeddings.
- (Newly required by Prof. Ding)

- Prof. Ding:
 - For each 5 category, pick one to show **the plot** of the predicted vector in 64 dimension space and the search result (in a 64 dimension vector).